



PRIMEIRA ENTREGA • 2026.2

# Organiza

Organizador de Trabalhos em Grupo

Disciplina: Projeto de Extensão em Web Back-End

Professor: Marcos

Integrantes: Isaac Azevedo Werneck e Tassia Carolina de Mello Costa

Data: 28 de setembro de 2026

# 1. Visão geral do projeto

O Organiza é uma aplicação web para universitários e pequenas equipes criarem grupos, compartilharem tarefas e acompanharem o trabalho concluído em um único lugar. A proposta prioriza rapidez, clareza e pouca configuração.

## Problema identificado

Trabalhos em grupo são frequentemente coordenados por mensagens dispersas e anotações separadas. Isso dificulta saber o que falta, quem já está no grupo e se todos estão vendo a mesma situação das tarefas.

## Público-alvo

- Principal: estudantes universitários que realizam trabalhos acadêmicos em grupo.
- Secundário: pequenas equipes que precisam de uma solução rápida para organizar atividades.

## Objetivo geral

Criar uma aplicação prática que permita a usuários autenticados montar um grupo, convidar outras pessoas por um código único e acompanhar tarefas compartilhadas.

## Equipe e divisão

Isaac Azevedo Werneck e Tassia Carolina de Mello Costa dividem igualmente análise, documentação, front-end, back-end, banco de dados, testes e apresentação.

### CRITÉRIO DE SUCESSO

**Dois usuários distintos conseguem entrar no mesmo grupo por código, visualizar a mesma tarefa e persistir sua alteração de estado.**

## 2. Escopo e requisitos

### Escopo do MVP

- Cadastro, login, sessão persistente e logout.
- Criação de grupo com geração automática de código de convite.
- Entrada de outro usuário no grupo por meio do código.
- Listagem de grupos, membros e tarefas compartilhadas.
- Criação de tarefa e alteração entre pendente e concluída.
- Validação das entradas e autorização por participação no grupo.

### Requisitos funcionais principais

| ID        | Requisito                                                               |
|-----------|-------------------------------------------------------------------------|
| RF01-RF04 | Cadastrar, autenticar, manter sessão e encerrar sessão.                 |
| RF05-RF09 | Listar e criar grupos, gerar convite, entrar por código e exibir grupo. |
| RF10-RF12 | Criar e listar tarefas e alterar seu estado.                            |

### Requisitos não funcionais

| Área         | Aplicação no projeto                                              |
|--------------|-------------------------------------------------------------------|
| Segurança    | Senha com hash seguro, cookie HttpOnly e autorização no back-end. |
| Usabilidade  | Ações principais em poucos passos e mensagens compreensíveis.     |
| Persistência | SQLite mantém usuários, grupos, tarefas e sessões.                |
| Manutenção   | Separação simples entre interface, servidor e banco.              |

### Fora do MVP

Recuperação de senha, permissões diferentes, responsáveis e prazos, comentários, anexos, notificações, chat, aplicativo móvel e integrações externas permanecem fora da primeira versão.

### 3. Regras de negócio e aceite

| Regra   | Descrição                                                                                        |
|---------|--------------------------------------------------------------------------------------------------|
| Usuário | Nome único; credenciais obrigatórias; senha nunca armazenada em texto puro.                      |
| Grupo   | Código de convite único; criador incluído automaticamente; participação sem duplicidade.         |
| Acesso  | Somente usuários autenticados e membros consultam ou alteram um grupo.                           |
| Tarefa  | Título obrigatório; pertence a um grupo; inicia pendente; qualquer membro pode alterar o estado. |

#### Critérios de aceite verificados

- Dois usuários diferentes conseguem criar contas; duplicidade de nome é rejeitada.
- O criador recebe um grupo com código único e passa a ser membro automaticamente.
- Outro usuário entra com o código correto; código inexistente e entrada duplicada são recusados.
- A tarefa nasce pendente, é vista por outro membro e mantém o novo estado após atualização.
- Um usuário de fora recebe resposta 403 ao tentar acessar o grupo.

#### Atores

| Ator                | Ações                                                  |
|---------------------|--------------------------------------------------------|
| Visitante           | Criar conta e entrar.                                  |
| Usuário autenticado | Ver, criar e entrar em grupos; encerrar sessão.        |
| Membro do grupo     | Ver participantes e tarefas; criar e concluir tarefas. |

# 4. Processo BPMN

O diagrama representa o fluxo completo entre usuário, sistema e SQLite: autenticação, criação ou entrada em grupo, criação de tarefa, conclusão e logout.

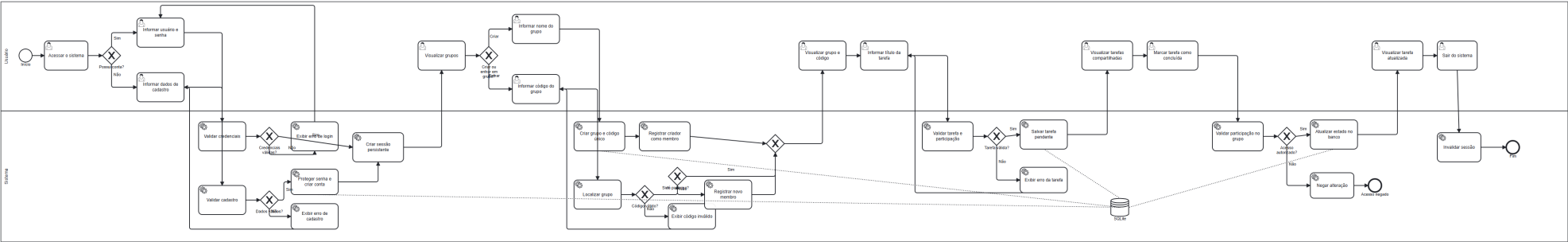


Figura 1 - Visão geral do processo Organizar trabalho em grupo.

| Etapa     | Fluxo principal                                   | Alternativa tratada                                          |
|-----------|---------------------------------------------------|--------------------------------------------------------------|
| Acesso    | Cadastro ou login cria sessão persistente.        | Dados ou credenciais inválidos retornam ao formulário.       |
| Grupo     | Usuário cria grupo ou entra por código.           | Código inválido gera nova tentativa; membro não é duplicado. |
| Tarefa    | Membro cria a tarefa pendente e depois a conclui. | Entrada inválida é recusada.                                 |
| Segurança | Participação é verificada antes da alteração.     | Acesso não autorizado é negado.                              |

## 5. Modelo do banco de dados

O SQLite foi escolhido por ser relacional, local e executável sem instalar um servidor separado. O projeto usa o módulo nativo `node:sqlite`.

| Tabela        | Função                   | Relações e controles                                 |
|---------------|--------------------------|------------------------------------------------------|
| users         | Contas dos usuários.     | username único; senha processada com script.         |
| groups        | Grupos de trabalho.      | Criador e invite_code único.                         |
| group_members | Relação usuários-grupos. | Chave composta evita participação duplicada.         |
| tasks         | Tarefas compartilhadas.  | Pertence a um grupo; estado 0/1 e data de conclusão. |
| sessions      | Sessões persistentes.    | Guarda hash do token, usuário e expiração.           |

### Relacionamentos

| Origem  | Cardinalidade         | Destino                   |
|---------|-----------------------|---------------------------|
| Usuário | 1:N                   | Grupos criados            |
| Usuário | N:N por group_members | Grupos em que participa   |
| Grupo   | 1:N                   | Tarefas                   |
| Usuário | 1:N                   | Sessões e tarefas criadas |

### Integridade

- PRIMARY KEY, FOREIGN KEY, UNIQUE, NOT NULL e CHECK reforçam as regras no próprio banco.
- ON DELETE CASCADE remove vínculos e tarefas dependentes quando necessário.
- PRAGMA foreign\_keys = ON é ativado em cada conexão.
- Índices atendem às consultas de grupos, tarefas e sessões.

## 6. Arquitetura e tecnologias

### Arquitetura cliente-servidor

| NAVEGADOR               | HTTP + JSON | SERVIDOR                  | SQL | BANCO                      |
|-------------------------|-------------|---------------------------|-----|----------------------------|
| HTML • CSS • JavaScript | →           | Node.js • Express         | →   | SQLite                     |
| Interface e interações  |             | Rotas, validação e regras |     | Persistência e integridade |

### Escolhas técnicas

| Tecnologia             | Uso e justificativa                                                   |
|------------------------|-----------------------------------------------------------------------|
| HTML, CSS e JavaScript | Recursos nativos, responsivos e suficientes para o MVP.               |
| Node.js 24+            | Uma linguagem no front e back; oferece sqlite, crypto e test nativos. |
| Express                | Única dependência; simplifica HTTP, JSON e arquivos estáticos.        |
| SQLite                 | Banco relacional local em um arquivo, com configuração mínima.        |

### Organização do código

| Pasta     | Conteúdo                                                      |
|-----------|---------------------------------------------------------------|
| public/   | Páginas, estilos e JavaScript do navegador.                   |
| src/      | Servidor, autenticação, regras e acesso ao banco.             |
| database/ | Esquema SQL, inicialização e teste do banco.                  |
| test/     | Teste integrado do fluxo da API.                              |
| docs/     | Requisitos, BPMN, banco, arquitetura, checklist e evidências. |

## 7. Implementação do MVP

### Rotas principais

| Método     | Rota                               | Função                              |
|------------|------------------------------------|-------------------------------------|
| POST       | /api/auth/register                 | Cadastrar e iniciar sessão.         |
| POST       | /api/auth/login                    | Autenticar usuário.                 |
| POST       | /api/auth/logout                   | Encerrar sessão.                    |
| GET        | /api/auth/me                       | Consultar sessão atual.             |
| GET / POST | /api/groups                        | Listar ou criar grupos.             |
| POST       | /api/groups/join                   | Entrar pelo código.                 |
| GET        | /api/groups/:groupId               | Consultar grupo, membros e tarefas. |
| POST       | /api/groups/:groupId/tasks         | Criar tarefa.                       |
| PATCH      | /api/groups/:groupId/tasks/:taskId | Alterar estado da tarefa.           |

### Segurança implementada

- Senhas derivadas com scrypt e salt individual.
- Token de sessão aleatório; somente seu hash fica no banco.
- Cookie HttpOnly, SameSite=Lax e Secure quando houver HTTPS.
- Consultas SQL parametrizadas e verificação de participação no servidor.
- Mensagens de login não revelam qual credencial está incorreta.



## 8. Evidências da interface

### Tela de acesso

O acesso concentra login e cadastro, com linguagem curta e indicação direta das funções do sistema.

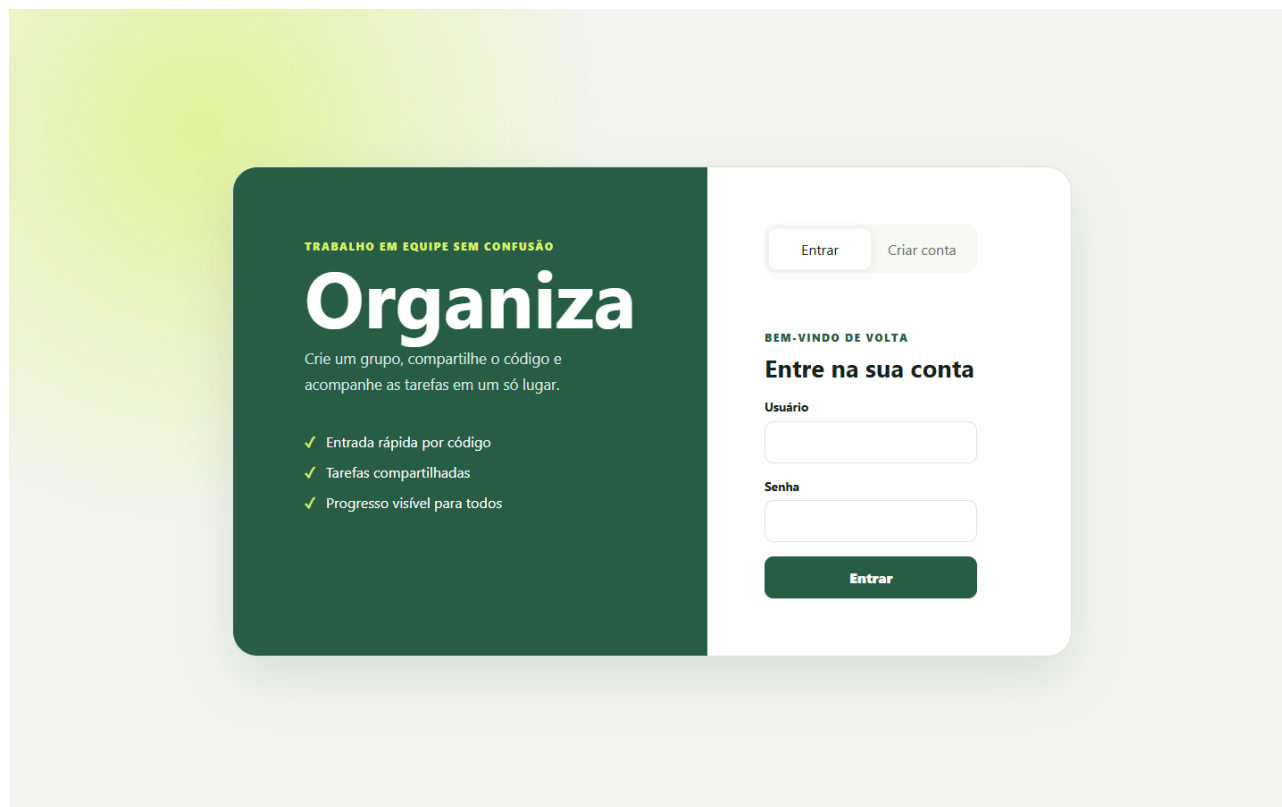


Figura 2 - Tela inicial do Organiza executada no navegador.

## 8. Evidências da interface (continuação)

### Painel de grupo e tarefas

A imagem comprova o grupo criado, o código de convite, o usuário autenticado e a tarefa persistida como concluída.

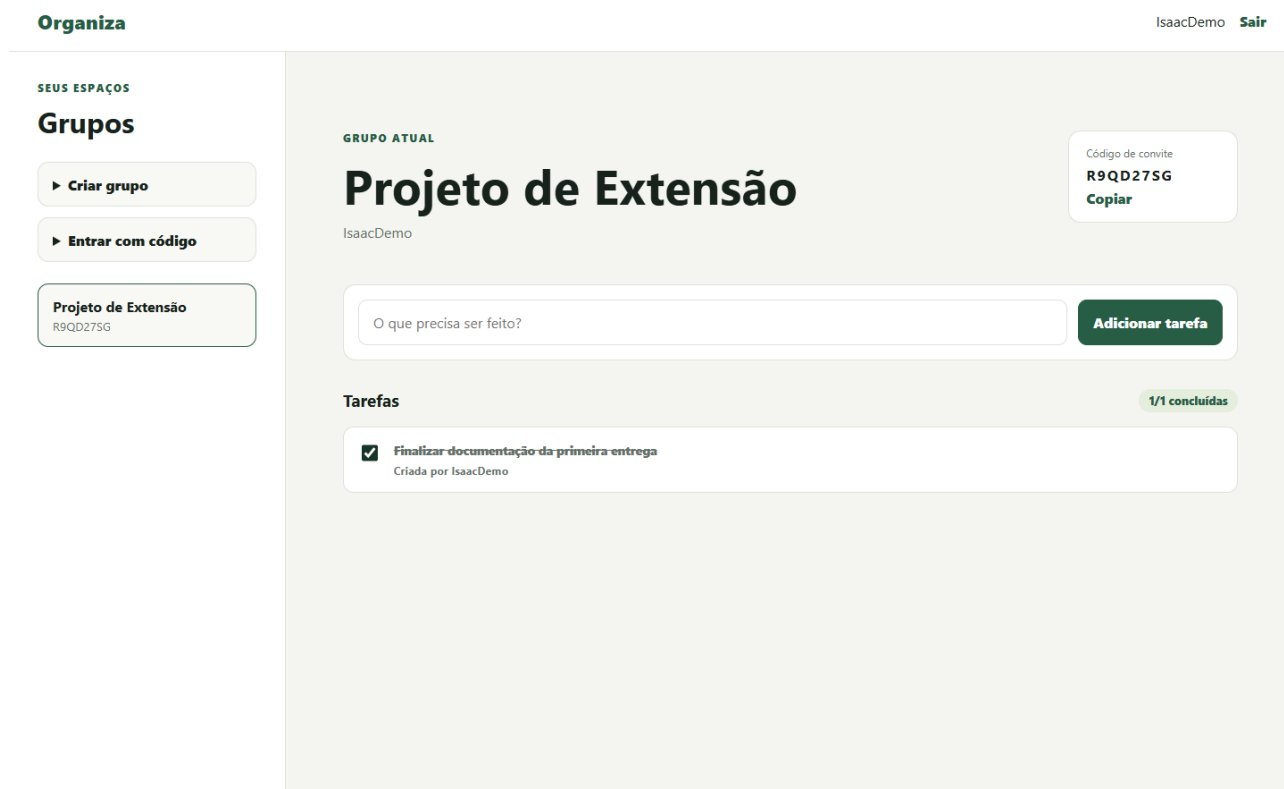


Figura 3 - Grupo Projeto de Extensão com uma tarefa concluída (1/1).

## 9. Verificação e resultados

### Teste automatizado

O comando `npm test` executa o teste do esquema e um fluxo integrado da API. Resultado final: 2 testes aprovados e 0 falhas.

| Cenário                       | Resultado                                                       |
|-------------------------------|-----------------------------------------------------------------|
| Criação do banco e restrições | Aprovado; integridade ok e chaves estrangeiras ativas.          |
| Cadastro e duplicidade        | Aprovado; contas distintas aceitas e nome repetido rejeitado.   |
| Grupo e convite               | Aprovado; criador incluído e segundo usuário entra pelo código. |
| Tarefa compartilhada          | Aprovado; outro membro visualiza e conclui a tarefa.            |
| Autorização                   | Aprovado; usuário externo recebe HTTP 403.                      |
| Logout                        | Aprovado; sessão deixa de autorizar novas operações.            |

### Validação manual

O fluxo foi repetido no navegador com dois usuários. Durante a verificação foram corrigidos a exibição de elementos ocultos e o momento de leitura dos formulários antes de desabilitar os controles.

### Resultado

O critério de sucesso foi atendido: usuários distintos compartilham um grupo e a mesma tarefa com persistência e controle de acesso.

# 10. Execução, repositório e conclusão

## Como executar

| Etapa          | Comando                                                                  |
|----------------|--------------------------------------------------------------------------|
| 1. Clonar      | git clone https://github.com/isaacwerneck/trabalho-extensao-back-end.git |
| 2. Instalar    | npm install                                                              |
| 3. Criar banco | npm run init-db                                                          |
| 4. Iniciar     | npm start                                                                |
| 5. Abrir       | http://localhost:3000                                                    |
| 6. Testar      | npm test                                                                 |

## Repositório

[github.com/isaacwerneck/trabalho-extensao-back-end](https://github.com/isaacwerneck/trabalho-extensao-back-end)

## Conclusão

A primeira versão do Organiza transforma o problema identificado em um sistema web funcional, documentado e testado. A solução entrega o núcleo necessário sem complexidade desnecessária: autenticação, grupos por convite, tarefas compartilhadas e persistência local.

## Próximas etapas após avaliação

- Validar o material com o professor e aplicar o feedback recebido.
- Avaliar responsáveis e prazos somente se forem necessários para a segunda entrega.
- Repetir os testes e revisar a documentação antes do envio final no AVA.

**MVP IMPLEMENTADO • DOCUMENTADO • TESTADO**